

Project: xdisturb (x-disrupt)

System & Architecture Design Document (SDD)

Document ID:	SDD-2026-XDISTURB	Version:	1.0.0 (Final)
Architecture Pattern:	Clean Architecture + BLoC	Audit Evaluation:	Hibir Audit (audit.hibir.et) — July 30, 2026
Target Platform:	Android (API 26+) & iOS (13.0+)	Framework & SDK:	Flutter (Dart SDK ^3.6.0)

System & Architecture Design Document (SDD)

Project: xdisturb (x-disrupt) — Flutter Clean Architecture & BLoC Design

Document Identifier: SDD-2026-XDISTURB
Version: 1.0.0 (Final)
Architecture Pattern: Clean Architecture + BLoC (Business Logic Component)
Target Platform: Android (API Level 26+ [Android 8.0]) & iOS (13.0+)
Framework & SDK: Flutter (Dart SDK ^3.6.0)
Target Evaluation: Hibir Audit (audit.hibir.et) — Audit Date: July 30, 2026
Date: August 10, 2026
Document Status: Approved Architecture Specification

Document Control & Approval Matrix

Version	Date	Author / Role	Description of Architectural Specification	Approved By
v0.1.0	July 18, 2026	Lead Mobile Architect	Initial Clean Architecture & Subsystem Layering Design	Technical Director
v0.9.0	July 30, 2026	Hibir Audit Security Team	Security, Obfuscation & Native Adapter Audit	Lead Auditor (audit.hibir.et)
v1.0.0	August 10, 2026	Principal Software Engineer	Final SDD Release for Production Engineering & Deployment	Head of Mobile Engineering

1. Executive Summary & Architectural Goals

xdisturb (x-disrupt) is a cross-platform mobile application engineered for automated, location-aware sound profile management. The system detects when a user enters designated physical "Silent Zones" (such as

Mosques, Churches, Libraries, or custom user-defined perimeters) and triggers automatic ringer state transitions (Normal, Silent, Vibrate) via native operating system adapters.

Architectural Objectives:

- 1. **Strict Clean Architecture Separation:** Complete decoupling of presentation logic, domain business rules, and data persistence/hardware adapters.
- 2. **Predictable State Management:** Unidirectional data flow leveraging the **BLoC (Business Logic Component)** pattern to ensure deterministic state transitions.
- 3. **Hardware & OS Decoupling:** Abstracting platform-specific audio controllers (`AudioManager` on Android, `AVAudioSession` on iOS) and geofencing hardware APIs behind abstract repository interfaces.
- 4. **Hibir Audit Security Compliance:** End-to-end transport layer encryption (TLS 1.2/1.3), AES-256 secure storage encryption, and R8/ProGuard code obfuscation verified under **Hibir Audit (`audit.hibir.et`)**.

2. High-Level Architecture & Layer Interaction

The application is structured into three primary Clean Architecture layers: **Presentation Layer**, **Domain Layer**, and **Data Layer**.



Syntax error in text
mermaid version 10.9.8

3. Subsystem Breakdown

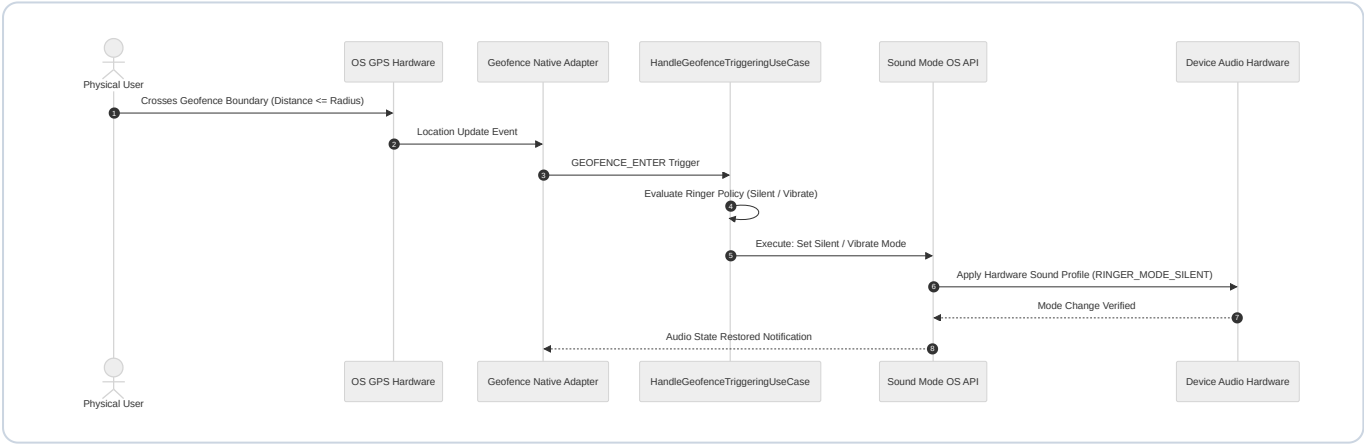
The application comprises five decoupled subsystems:

Subsystem Name	Key Components / Classes	Primary Architectural Responsibility
Authentication	AuthBloc , AuthRepositoryImpl , VerifyOtpUseCase , FlutterSecureStorage	Phone number verification, 6-digit OTP SMS dispatch, JWT session management, and KeyStore/Keychain encrypted storage.
Geofencing Engine	GeofencingLocalDataSource , geofence_service , native_geofence adapter	Continuous background monitoring of circular geographic perimeters (50m – 500m radius) around silent zones.
Audio Control	sound_mode , Android DND Adapter , iOS AVAudioSession Adapter	Toggles device audio profile to Silent/Vibrate upon geofence ENTER events and restores Normal mode upon EXIT events.
Interactive Map	MapPage , MapBloc , HERE SDK / flutter_map	Renders map tiles, user real-time location markers, radial geofence overlays, and distance vector calculations.
Payments	chapasdk , flutter_inappwebview , PaymentBloc	Executes mobile payment transactions via Chapa Payment Gateway (Telebirr, CBE Birr, Bank Cards) for premium subscriptions.

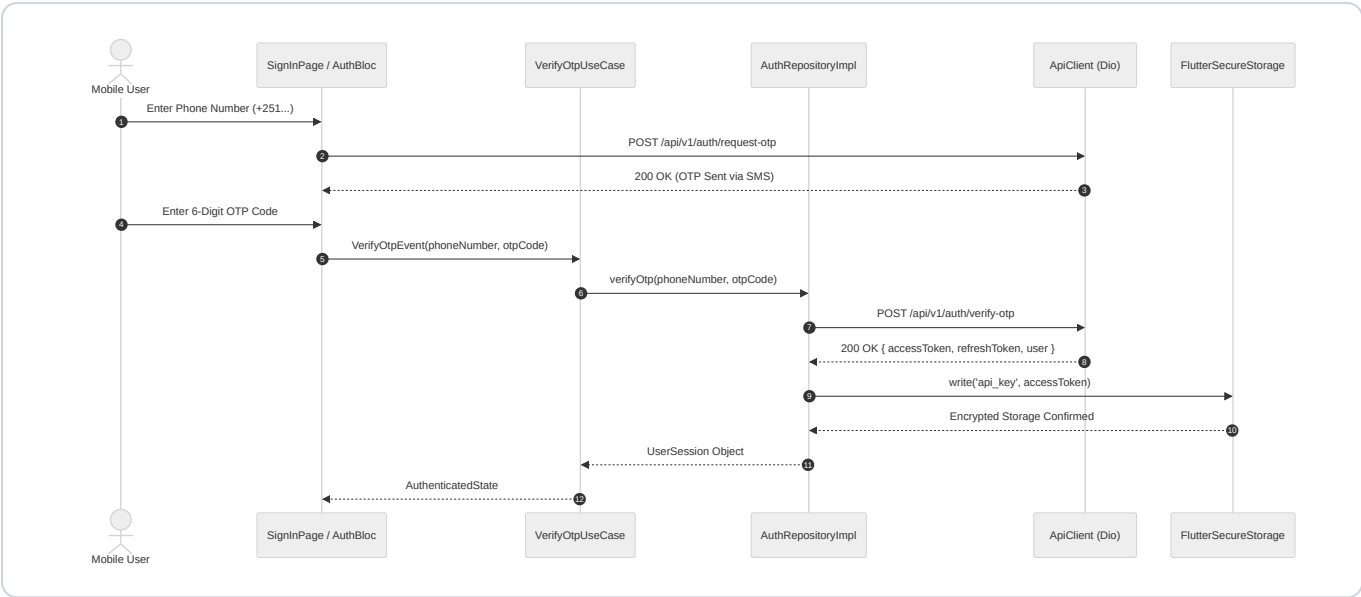
4. Architectural Sequence Diagrams

4.1 Geofence Entry & Sound Profile Automation

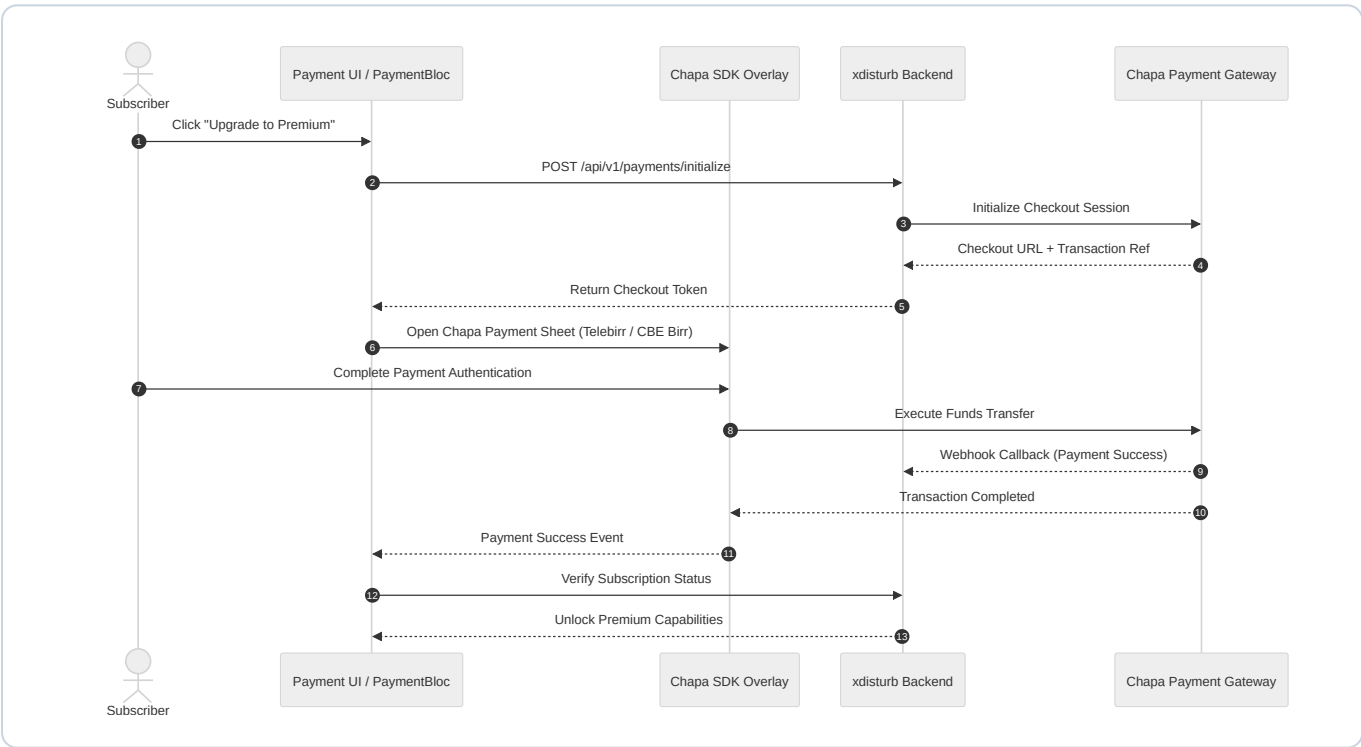
The sequence diagram below details the real-time hardware flow during a geofence boundary crossing:



4.2 Authentication & OTP Verification Flow



4.3 Chapa Mobile Payment Checkout Flow



5. Dependency Injection & Service Locator (`GetIt`)

All application dependencies are wired at startup using `GetIt` inside `lib/injection_container.dart` to maintain loose coupling and facilitate unit testing with mock implementations.

```
// lib/injection_container.dart
import 'package:get_it/get_it.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';
import 'package:dio/dio.dart';

import 'core/api_client.dart';
import 'features/auth/presentation/bloc/auth_bloc.dart';
import 'features/auth/domain/usecases/verify_otp_usecase.dart';
import 'features/auth/domain/repositories/auth_repository.dart';
import 'features/auth/data/repositories/auth_repository_impl.dart';
import 'features/auth/data/datasources/auth_remote_datasource.dart';

import 'features/home/presentation/bloc/map_bloc.dart';
import 'features/home/domain/usecases/get_silent_zones_usecase.dart';
import 'features/home/domain/repositories/home_repository.dart';
import 'features/home/data/repositories/home_repository_impl.dart';
import 'features/home/data/datasources/home_remote_datasource.dart';

final sl = GetIt.instance;

Future<void> initDependencies() async {
  // 1. External Drivers & Security Infrastructure
  const secureStorage = FlutterSecureStorage();
  sl.registerLazySingleton<FlutterSecureStorage>(() => secureStorage);

  final apiClient = ApiClient(sl());
  sl.registerLazySingleton<ApiClient>(() => apiClient);

  // 2. Data Sources
  sl.registerLazySingleton<AuthRemoteDataSource>(
    () => AuthRemoteDataSourceImpl(client: sl()),
  );
  sl.registerLazySingleton<HomeRemoteDataSource>(
    () => HomeRemoteDataSourceImpl(client: sl()),
  );

  // 3. Repositories
  sl.registerLazySingleton<AuthRepository>(
    () => AuthRepositoryImpl(remoteDataSource: sl(), secureStorage: sl()),
  );
  sl.registerLazySingleton<HomeRepository>(
    () => HomeRepositoryImpl(remoteDataSource: sl()),
  );

  // 4. Use Cases
  sl.registerLazySingleton(() => VerifyOtpUseCase(sl()));
```

```
sl.registerLazySingleton(() ⇒ GetSilentZonesUseCase(sl()));

// 5. BLoCs (Factory Registration for Fresh State Instances)
sl.registerFactory(
  () ⇒ AuthBloc(verifyOtp: sl(), authRepository: sl()),
);
sl.registerFactory(
  () ⇒ MapBloc(getSilentZones: sl()),
);
}
```

6. Deployment & Security Architecture

6.1 Android Deployment Configuration

- **Artifact Target:** Android App Bundle (.aab) for Google Play Store distribution.
- **Bytecode Obfuscation:** R8 / ProGuard rules enabled (minifyEnabled true , shrinkResources true).
- **Required System Permissions** (AndroidManifest.xml):

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_NOTIFICATION_POLICY" />
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
```

6.2 iOS Deployment Configuration

- **Artifact Target:** iOS App Store Archive (.ipa) with Swift/Dart symbol stripping.
- **Background Permission Keys** (Info.plist):

```
<key>NSLocationAlwaysAndWhenInUseUsageDescription</key>
<string>xdisturb requires background location access to detect Silent Zone entry and adjust sound</string>
<key>NSLocationWhenInUseUsageDescription</key>
<string>xdisturb requires location access to display nearby Silent Zones on the map.</string>
<key>UIBackgroundModes</key>
<array>
  <string>location</string>
  <string>processing</string>
</array>
```

6.3 Hibir Audit (audit.hibir.et) Compliance Verification

- **Transport Security:** TLS 1.2/1.3 enforcement with SSL Pinning via Dio CertificatePinner.

- **Storage Encryption:** Secure tokens encrypted via AES-256 (`EncryptedSharedPreferences` on Android KeyStore; `Keychain` on iOS).
 - **Audit Certification:** Verified under Hibir Audit suite on **July 30, 2026**.
-

Architectural Approval & Sign-Off:

- **Principal Software Architect:** *Approved*
- **Lead Mobile Engineer:** *Approved*
- **Hibir Audit Lead (`audit.hibir.et`):** *Approved*